UNIDAD #3: DESARROLLO DE SISTEMAS.

Este punto debe ser el puente donde el papel (diseño) se convierte en realidad (código) de forma controlada, ya que un sistema "implantable" es aquel que es fácil de leer y difícil de romper, esto es mucho más sencillo de ejecutar, gracias a la estructura definida en el diseño, lo cual permitirá a los desarrolladores un camino claro, hacia la construcción de un software, sin embargo, para conservar la coherencia y la cordura durante el desarrollo, se deben tener en cuenta los siguientes aspectos:

3.1 Contrato Técnico.

El contrato técnico es el acuerdo inamovible entre el Diseño y la Construcción, es lo que garantiza que las piezas encajen cuando el sistema se implante, sin embargo, en ocasiones se deben hacer pequeños ajustes para que la implantación sea lo más realista posible, por lo cual, si se necesita hacer algún ajuste en el diseño, esto debe estar respaldado por una razón significativa.

- **Definición de Interfaces (APIs):** Antes de programar la lógica, se deben definir las "entradas y salidas", si el Frontend espera un JSON con la variable user_id, el Backend no puede enviar idUsuario.
- **Tipado Estricto y DTOs (Data Transfer Objects):** Los datos entre módulos deben viajar en estructuras definidas, para evitar la sobreescritura de código y mantener un estándar de comunicación definido entre los componentes.
- **Protocolos de Comunicación:** Definir cómo se comunicarán las partes (ej. REST, GraphQL). Si esto no se respeta, cuando se intente unir el sistema, nada funcionara.
- **Cumplimiento del Esquema de Datos:** El código debe ser un reflejo exacto del Diagrama Entidad-Relación, si el diseño dice "No Nulo", el código debe validar eso antes de intentar guardar en la base de datos, siguiendo siempre la estructura de base de dato que hayan definido a utilizar, sea relacional o no relacional.

Una vez que el contrato técnico (API) se firma, no se cambia unilateralmente. Si el Backend cambia un campo sin avisar, “rompe” al Frontend, en la mayoría de los casos la implantación falla por falta de comunicación.

3.2 Paridad de entornos.

Este concepto es el corazón de la implantación moderna, se basa en que el entorno donde se desarrolla debe ser idéntico (o lo más parecido posible) al entorno donde se va a desplegar.

- **Aislamiento de Configuración:** Las rutas de archivos, contraseñas de BD y puertos **nunca** deben estar escritos directamente en el código (*hardcoded*), para esto utilizamos algo que se conoce como variables de entorno.
- **El archivo .env:** Es la herramienta clave, es en donde se almacenan esas variables de entorno aquellas que contienen la información necesaria para la ejecución del sistema, el código lee variables, por lo tanto, los valores reales viven en el archivo .env. Por lo tanto, subir un .env con credenciales a GitHub no es un error, es una vulnerabilidad de seguridad crítica que puede costar miles de dólares o el robo de datos.
- **Contenedores:** Existen herramientas como Docker estas permiten empaquetar todo (código, librerías, sistema operativo) en una "caja" que se comporta igual en cualquier computadora, indiferentemente de donde se despliegue.
- **Gestión de Dependencias:** No reinventamos la rueda. Usamos librerías externas, pero debemos controlarlas, existen archivos al inicializar un proyecto, que nos permite saber que componentes externos usa nuestro sistema.
- **El Proceso de "Build":** En el desarrollo profesional, el código que escribimos (Source) no es el que se ejecuta (Artifact), En producción solo enviamos el resultado del build, no todo el desorden de carpetas del desarrollo.

3.3 Calidad y estándares defensivos.

En ocasiones se piensa que un sistema profesional no es el que no tiene errores, pero realmente es aquel que sabe qué hacer cuando algo falla.

- **Validación de Entradas:** "Nunca confíes en el usuario", si el sistema espera un número, debe validar que no reciba texto antes de procesarlo, esto aplica para todo, principalmente sanear los inputs de toda la aplicación, para evitar inyecciones de código malicioso.
- **Logging Profesional:** No usar print o console.log solo para ver si algo funciona o no, lo recomendable es escribir archivos de registro (Logs) que indiquen: Fecha, Hora, Nivel de error (Info/Warning/Error) y Mensaje, esto es vital para dar soporte tras la implantación.
- **Linters y Formateadores:** Obligar al uso de herramientas (como Prettier) que automaticen la estética del código, el código debe parecer escrito por una sola persona, sin importar cuántos desarrolladores participen.
- **Principios SOLID:**
    - **S - Single Responsibility Principle (SRP - Principio de Responsabilidad Única):** Una clase debe tener una sola razón para cambiar, lo que significa que debe tener una única responsabilidad.
    - **O - Open/Closed Principle (OCP - Principio de Abierto/Cerrado):** Las entidades de software (clases, módulos) deben estar abiertas para la extensión, pero cerradas para la modificación.
    - **L - Liskov Substitution Principle (LSP - Principio de Sustitución de Liskov):** Las clases derivadas deben poder sustituir a sus clases base sin alterar el funcionamiento correcto del sistema.
    - **I - Interface Segregation Principle (ISP - Principio de Segregación de Interfaces):** Es preferible tener muchas

interfaces específicas que una sola interfaz general, ninguna clase debe depender de métodos que no utiliza.

- **D - Dependency Inversion Principle (DIP - Principio de Inversión de Dependencias):** Los módulos de alto nivel no deben depender de los de bajo nivel; ambos deben depender de abstracciones.

3.4 Trazabilidad y control de versiones.

Git no es un servicio de almacenamiento; es una herramienta de ingeniería para rastrear la evolución del sistema, lo cual es un aporte significativo a la documentación.

- **Flujo de Trabajo (GitFlow):** Separar lo que se está "construyendo" de lo que ya está "listo para entrega".
  - **main:** Solo código estable y probado.
  - **develop:** El día a día de la integración.
  - **feature/:** Ramas para cada funcionalidad específica.
- **Conventional Commits:** Estandarizar cómo se describen los cambios un commit que dice "arreglé algo" es basura técnica y no documenta nada, un commit que dice “fix(auth): corregir error de sesión expirada” permite trazabilidad real cuando el sistema falle en producción.
- **Documentación Evolutiva:** No se documenta al final; se comenta el código y se actualiza el README mientras se desarrolla. Un código sin instrucciones de instalación no se puede implantar.
- **Pull Requests (PR):** El control de versiones incluye la revisión por pares, nadie es dueño absoluto del código; la calidad es responsabilidad del equipo, esto es fundamental, ya que permite subir código para su verificación y aceptación según sea la rama en cuestión.
- **El archivo .gitignore:** este archivo permite indicarle a git, que es lo que no se desea subir al repositorio una vez se haga un PR a una rama determinada.

3.5 Definicion de hecho (Definition of Done DoD).

Un requerimiento no está "terminado" porque el código compile, está terminado cuando:

- Cumple con el **Contrato Técnico** (API y DB).
- Pasa el **Linter** y el formateo.
- Se ha realizado un **Pull Request** y ha sido revisado.
- Está documentado en el **README**.
- Funciona en un entorno limpio usando solo el archivo **.env**.

